IWACU RWANDA

Civic Geospatial Platform for Rwanda

Complete Technical Project Specification

Concept · Architecture · Implementation · Current State · Definition of Done

This document is written so that any software engineer can understand what the product is, how it is built, what is already done, what remains, and how to rebuild or continue the work from a single source of truth.

Field	Value
Document type	Technical specification / handoff
Product name	Iwacu Rwanda
UI languages	Kinyarwanda (primary), English (secondary)
Live URL	https://iwacu-rwanda.vercel.app
Code repository	github.com/eddyizere24-collab/Iwacu-rwanda- (branch stage1-clean)
Design reference package	iwacu-rwanda-stage1-v3.zip
Document date	28 October 2026
Audience	Software developers, technical partners, auditors

1. Vision and problem

People in Rwanda often need to know exactly which administrative unit they are in (village, cell, sector, district) and who their local leaders are, with a practical way to contact them. Paper lists and informal channels fail for newcomers, travellers, and emergency situations.

Iwacu Rwanda is a civic Progressive Web App that answers from a phone browser:

- Where am I in the official hierarchy (GPS to village, cell, sector, district)?
- How confident is that answer (GPS accuracy versus verified polygons)?
- Who are the verified, current leaders for those units?
- How do I reach them (phone, WhatsApp, email, other public channels)?
- What emergency numbers work even offline?

1.1 Design principles

- **Nationwide from day one** — schema and code are not hard-coded to one district.
- **Citizens stay anonymous** — no citizen accounts, no profiles, no stored location history.
- **Leader contacts are public by design** — so any citizen can reach them.
- **Authoritative data only** — never invent boundary polygons; never present fabricated people as real leaders.
- **Privacy-aware** — aligned with responsible handling under Rwanda data-protection expectations.
- **Offline resilience** — app shell and emergency list offline; optional sector packs with Turf.js.

1.2 What 100% complete means

Public launch is justified only when all of the following hold:

- Production serves the verified stage-1 codebase with healthy checks (env + database reachable).
- PostGIS holds verified boundary data for the agreed launch scope (nationwide architecture; data may be phased).
- Locate returns ranked confidence and hierarchy inside Rwanda; clear errors for outside, gaps, low confidence.
- Verified current leaders and public contacts appear for matched scopes.
- National emergency numbers always visible; local rows when in scope.
- Admins sign in via magic link and edit only within authorised scope (RLS + audit).
- Automated tests pass in CI; stage-1 security items closed.

2. User flows

2.1 Citizen (public)

1. Open the home page. The emergency bar shows national numbers immediately (also available offline from static JSON).
2. Request location; the browser Geolocation API returns latitude, longitude, and accuracy in metres.
3. Client sends **POST /api/locate** with those three fields (rate-limited per IP).
4. Server validates the body (Zod), rejects excessive accuracy, prefilters outside Rwanda, calls PostGIS RPCs (point_in_rwanda, calculate_location_confidence), builds hierarchy from parent links, loads verified leaders and emergency rows, returns JSON.
5. UI renders hierarchy, confidence, and leader cards with actionable contact links.

2.2 Administrator

1. Open /admin/login, submit email, receive Supabase Auth magic link (shouldCreateUser is false — the user must already exist).
2. After /auth/callback, land on /admin/dashboard. Middleware enforces a session when Supabase env is configured.
3. Dashboard lists leaders in scope. Edits use optimistic locking (version column; HTTP 409 on conflict).
4. Writes go through /api/admin/leaders; database audit triggers record changes.

3. Architecture

3.1 Technology stack

Layer	Technology	Notes
Frontend	Next.js 15 App Router, React 19, Tailwind, Lucide	PWA: manifest + service worker
API	Next.js Route Handlers (Node runtime)	locate, leaders, emergency, health, admin, ai-router
Database	Supabase PostgreSQL + PostGIS + ltree	RLS; geospatial RPCs
Auth	Supabase Auth (email magic link)	Admins only
Rate limiting	Upstash Redis REST	Required in production for sensitive routes
Offline geo	Turf.js + IndexedDB	Leaders not shown offline
Optional AI	Groq function-calling only	No PII or leader payloads to the model
Hosting	Vercel	Git-linked to GitHub
CI	GitHub Actions	lint, typecheck, unit tests

3.2 Administrative hierarchy

country (RW) → province → district → sector → cell → village. Scope codes and parent links model the tree; PostGIS stores polygon geometry. Locate intersects a circle of radius equal to GPS accuracy with verified village polygons and returns confidence percentages.

3.3 Core endpoints

Endpoint	Method	Purpose
/api/health	GET	JSON health and env checks
/api/locate	POST	GPS → hierarchy, confidence, leaders, emergency
/api/leaders	GET	Public verified current leaders by scope
/api/emergency	GET	Emergency contacts
/api/admin/leaders	GET/PATCH	Scoped admin reads and writes; 409 on version conflict
/api/ai-router	POST	Optional intent routing (no personal data)
/auth/callback	GET	Auth code exchange → admin dashboard
/admin/login	page	Magic-link form
/admin/dashboard	page	Authenticated admin UI

3.4 Environment variables

Name	Visibility	Role
NEXT_PUBLIC_SUPABASE_URL	Public	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public	Anon key (RLS applies)
SUPABASE_SERVICE_ROLE_KEY	Server only	Privileged access; never ship to the browser
UPSTASH_REDIS_REST_URL	Server	Rate limiting
UPSTASH_REDIS_REST_TOKEN	Server	Rate limiting
NEXT_PUBLIC_APP_URL	Public	Canonical origin for redirects / CSRF
GROQ_API_KEY	Server optional	AI router

Full template: .env.example — never commit real secrets.

4. Data model

SQL lives under sql/migrations/ and is bundled as sql/apply_all.sql for one-shot apply in the Supabase SQL editor. Required extensions: postgis, ltree.

Object	Role
boundaries	Polygons, names, verification_status, parent links
leaders	People per scope_code and role; validity window; version for optimistic lock
admin_profiles	Auth user → allowed scope for RLS
contacts / contact kinds	Extensible public contact channels
emergency contacts	National and local numbers with source metadata
audit_logs	Append-only admin mutation trail
RPCs	point_in_rwanda, calculate_location_confidence, ancestor helpers

Public APIs expose only verified and current leaders. Pending rows stay hidden.

4.1 Confidence algorithm (summary)

Buffer a circle from (lat, lng, accuracy_m); intersect with verified village polygons; confidence ≈ intersection area / circle area; rank matches; reject below threshold (for example 5%); walk parents via stored links — never invent hierarchy.

5. Repository layout

```
src/app/ — pages and API routes
src/components/ — UI including admin forms
src/hooks/ — e.g. useLocation
src/lib/ — contacts, geo, security, supabase, turf, errors
src/types/ — shared TypeScript types
src/middleware.ts — admin dashboard session gate
sql/migrations/, sql/seed/, sql/apply_all.sql
tests/ — unit and Playwright e2e
public/ — manifest, service worker, emergency JSON, icons
scripts/ — boundary ingest, SQL bundle builder
docs/ — STATUS, ROADMAP, security and deployment notes
```

6. Current state (28 October 2026)

6.1 Live today

- https://iwacu-rwanda.vercel.app returns HTTP 200 with the stage-1 Next.js shell.
- GitHub branch stage1-clean is the integration branch used by Vercel project iwacu-rwanda.
- /api/health returns JSON; status is degraded until Supabase (and Redis) env vars are set.
- Admin login page is reachable; full authentication needs Supabase Auth.
- Vercel build has reached READY after TypeScript and App Router fixes.

6.2 Not done yet

- Supabase env on Vercel (URL, anon, service_role) — blocks locate and admin auth.
- SQL applied on a real database with postgis and ltree.
- Authoritative boundary data ingested and verified.
- Real leader data (or clearly flagged test-only leaders in non-production).
- Full byte parity of every file in stage1-v3.zip with GitHub (zip remains design reference).
- Production rate limiting via Upstash.

6.3 Do not confuse repositories

Repository	Role
Iwacu-rwanda-	Primary project (stage1-clean → production)
Iwacu-rwanda-2	Separate experiment; not production source of truth

6.4 Collaboration

Stage-1 design and the v3 package were structured with Claude (code, tests, SQL, docs, MANIFEST). GitHub/Vercel integration, production aliasing, and middleware that does not crash when env is missing were advanced with Grok. Creating the Supabase project and pasting secrets remain human steps.

7. Build, test, deploy

7.1 Local

```
cp .env.example .env.local
npm install
npm run dev
npm run typecheck && npm run lint && npm test && npm run build
```

7.2 Database

Create Supabase project → enable postgis and ltree → run sql/apply_all.sql (or migrations 001, 002, 003 and seed) → ingest boundaries only from authoritative sources → verify before public read.

7.3 Vercel

Link GitHub repo → set env for Production and Preview → deploy stage1-clean (or merge to main after green checks, no force-push) → confirm GET /api/health is healthy.

7.4 Admin bootstrap

Create Auth user → set Site URL and redirect URLs → insert admin_profiles with scope → sign in at /admin/login.

8. Security and privacy

- No citizen accounts; do not store GPS trails as a product feature.
- RLS and scope checks on admin data access.
- Service role key only on the server.
- Rate-limit locate and admin writes; production should not silently disable limits.
- AI path must not receive personal or leader PII.
- Append-only audit log for admin mutations.

See SECURITY_AUDIT.md in the repository for the stage-1 checklist.

9. Errors surfaced to citizens

Stable machine codes map to Kinyarwanda and English copy (GPS denied, accuracy too low, outside Rwanda, point in gap, confidence too low, rate limit, database error, and related cases). Keep codes stable; user-facing strings live in UI components such as ErrorDisplay.

10. Roadmap after stage 1

Stage	Focus
Stage 1	Nationwide-ready code, locate, emergency, admin, tests, deploy path
Stage 2	Authoritative boundary load and verification at scale
Stage 3	Admin UI for roles and contact kinds
Stage 4+	Richer offline packs, monitoring, formal security review

11. Rebuild from zero (engineer checklist)

1. Clone Iwacu-rwanda- (stage1-clean) and/or unpack iwacu-rwanda-stage1-v3.zip; align with MANIFEST if you need parity.
2. Use Node 20+; npm install; configure .env.local; run typecheck, tests, and build.
3. Create Supabase; enable postgis and ltree; apply SQL; configure Vercel env; redeploy.
4. Seed emergency national data; ingest and verify boundaries from official sources only.
5. Create admin Auth user and admin_profiles; test magic link and a scoped edit.
6. Call locate on a known point inside a verified polygon; confirm hierarchy and leaders.
7. Only then treat production as a launch candidate.

12. Definition of Done

#	Criterion	Status
1	Repo matches agreed stage-1 source (zip or locked SHA)	In progress

#	Criterion	Status
2	Vercel production READY on that line of code	Yes (app shell)
3	Required env set; /api/health not degraded	No
4	SQL applied; RPCs and tables exist	No
5	Locate works on real verified geometry	No
6	Emergency bar online + offline copy	Partial
7	Admin magic link + scoped edit + audit	No
8	CI green (typecheck, lint, unit tests)	Target
9	No fabricated leaders in public API	Policy — enforce in data
10	Stage-1 security items reviewed	Target

13. Glossary

Term	Meaning
Locate	POST API turning GPS into hierarchy and leaders
Scope code	Stable identifier of an administrative unit
Verified	Approved for public exposure
Magic link	Passwordless email login for admins
RLS	Row Level Security in Postgres
PWA	Installable web app with offline shell
stage1-clean	Git branch for stage-1 integration

End of specification. When this document and the repository differ on implementation detail, prefer the repository and the stage1-v3 zip; this document states product intent, architecture, and operational status as of the document date.